

实验报告

课程：数字逻辑与计算机组成实验

实验九：单周期 CPU

姓名： 郑袭明

学号： 251502027

日期：2026 年 5 月 22 日

目录

1 设计思路	2
1.1 整体架构	2
1.2 算术逻辑单元 (ALU) 控制设计	2
1.3 斐波那契程序机器码与执行流分析	3
2 实验总结与反思	5

1 设计思路

1.1 整体架构

本实验设计并实现了一个基于 RISC-V RV32I 指令集的单周期 CPU。该 CPU 能够在一个时钟周期内完成一条指令的取指 (Instruction Fetch)、译码 (Decode)、执行 (Execute)、访存 (Memory Access) 和写回 (Write Back)。

单周期 CPU 与指令存储器、数据存储器以及外部输入输出设备的连接关系如图 1 所示：

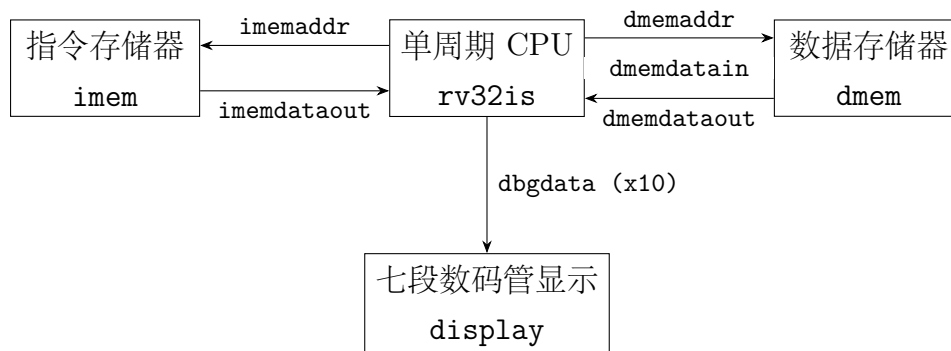


图 1: 单周期 CPU 系统整体架构图

在该设计中：

1. **取指阶段**：根据当前 PC 值从指令存储器 (ROM) 中读取 32 位机器指令。
2. **译码阶段**：控制器分析指令的 Opcode、funct3 和 funct7，生成寄存器写使能、内存写使能、操作数选择信号、ALU 操作控制码等控制信号。
3. **执行阶段**：通用寄存器堆 (Regfile) 读出操作数，经多路选择器送入 ALU。ALU 完成算术逻辑运算、分支条件判断或访存地址计算。
4. **访存阶段**：若为访存指令，则根据 ALU 计算出的地址对数据存储器 (RAM) 进行读或写。
5. **写回阶段**：将 ALU 计算结果或从内存读出的数据写入通用寄存器堆的目标寄存器。同时，PC 控制逻辑根据当前指令类型及分支判断信号决定下一周期 PC 的值。

1.2 算术逻辑单元 (ALU) 控制设计

根据 alu.v 的实现，本实验中的 ALU 通过 4 位控制信号 ALUctr 支持加法、减法、逻辑移位、算术右移、按位与/或/异或以及比较等多种运算。其设计核心包括：

- 分支决策信号：

- less 信号：在有符号/无符号比较时，根据控制信号最高位 `ALUctr[3]` 选择有符号比较 `$signed(dataa) < $signed(datab)` 或无符号比较 `dataa < datab`。
- zero 信号：在控制信号低三位为 `3'b010` 时（对应比较/分支操作），zero 信号表示两数是否相等（即 `dataa == datab`），这为分支跳转指令（如 `beq / bne`）提供了直接的判断标志；而在其他普通运算下，zero 信号表示运算结果 `alurestult` 是否为 0。

- 移位位数的处理：移位操作数取 `datab[4:0]`，实现了符合 RISC-V 规范的截断移位。

1.3 斐波那契程序机器码与执行流分析

程序 `fib.coe` 实现了一个用于计算斐波那契数列第 n 项（以 $n = 10$ 为默认输入）的循环程序。每一行机器码对应的反汇编指令及其功能如表 1 所示。

寄存器分配与状态说明

- `x1 (ra)`：函数返回地址寄存器，存放主程序的 `0x08`。
- `x10 (a0)`：参数与返回值寄存器。程序开始时保存待求的项数 n ，随后用来保存当前迭代的斐波那契数值 $Fib(i)$ ，最终返回时亦存放计算结果，并通过 `dbgdata` 输送到数码管显示模块。
- `x12 (a2)`：循环上限保存寄存器，值为 $n + 1$ 。
- `x13 (a3)`：中间数据暂存寄存器，用来保存计算过程中的中间项 $Fib(i - 1)$ 。
- `x14 (a4)`：迭代项 $Fib(i - 2)$ 的存放寄存器。
- `x15 (a5)`：循环变量 i 的存放寄存器。

板级控制逻辑设计 在板级验证时，顶层模块 `top` 实现了对程序流的两个重要外部干预：

1. **动态输入捕获**：当 CPU 指令计数器取到第一条指令 `0x00A00513` 时，顶层模块通过多路选择器将其替换为 `addi x10, x0, SW` 指令（其中 `SW` 为板载 8 位拨码开关的状态）。这允许通过拨码开关动态地给程序传入计算参数 n （支持 0 到 255）。

表 1: Fibonacci 程序的机器码与反汇编指令

地址 (PC)	机器码 (Hex)	汇编指令	功能说明
0x00	00A00513	addi x10, x0, 10	默认设置输入 $n = 10$ 到寄存器 x10, 在板级验证中由顶层模块改写为读取拨码开关 SW 的值
0x04	008000EF	jal x1, 12	调用计算函数, 跳转至 0x0C, 并将返回地址 0x08 存入 x1
0x08	DEAD10CC	dead10cc	停机特征码。顶层模块检测到该指令后使 CPU 挂起, 持续供给 NOP 指令, 锁定显示
0x0C	02050E63	beq x10, x0, 72	若 $n == 0$ (x10 == 0), 分支跳转至 0x48 直接返回
0x10	00100793	addi x15, x0, 1	设置辅助寄存器 x15 值为 1
0x14	02F50A63	beq x10, x15, 72	若 $n == 1$ (x10 == 1), 分支跳转至 0x48 直接返回
0x18	02A7D663	bge x15, x10, 68	若 $1 \geq n$ (处理异常负数输入), 跳转至 0x44 返回值清零后返回
0x1C	00150613	addi x12, x10, 1	设置循环次数上限: $x12 = n + 1$
0x20	00000713	addi x14, x0, 0	初始化第一项 $Fib(0) = 0$ 存入寄存器 x14
0x24	00078513	addi x10, x15, 0	初始化第二项 $Fib(1) = 1$ 存入寄存器 x10
0x28	00200793	addi x15, x0, 2	初始化循环计数器 $i = 2$ 存入 x15
0x2C	00050693	addi x13, x10, 0	循环体起点: 将当前的 $Fib(i-1)$ (在 x10 中) 存入暂存寄存器 x13
0x30	00178793	addi x15, x15, 1	循环计数器自增: $i = i + 1$
0x34	00A70533	add x10, x14, x10	计算 $Fib(i) = Fib(i-2) + Fib(i-1)$ 并更新到 x10
0x38	00068713	addi x14, x13, 0	更新 $Fib(i-2)$ 为旧的 $Fib(i-1)$ (将 x13 赋给 x14)
0x3C	FEF618E3	bne x12, x15, 44	若循环计数器未达到上限 $n + 1$ (即 $x12 \neq x15$), 则跳转回 0x2C 再次执行循环
0x40	00008067	jalr x0, x1, 0	正常计算结束, 函数返回, 跳转回 x1 (0x08)
0x44	00000513	addi x10, x0, 0	异常处理: 设定返回值 $x10 = 0$
0x48	00008067	jalr x0, x1, 0	返回跳转, 跳转回 x1 (0x08)

2. **停机状态机**：当程序执行完 `jalr` 返回到 `0x08` 时，取回的指令为自定义的停机码 `32'hdead10cc`。顶层模块的状态机捕获此码后，将 `halted` 置为高电平。此时，顶层模块会将送入 CPU 的指令强制替换为 `nop` 指令（即 `addi x0, x0, 0`），从而使得 PC 持续空转，CPU 内部的寄存器状态被锁定，达到了停机的效果，且能够长期稳定地在数码管上显示第 n 项的斐波那契数值。

2 实验总结与反思

在实验中，我最开始没有想到怎么处理这类程序的输入问题，此时该程序只能输出默认的 10 项的斐波那契数值。后来 AI 提醒我使用了这样一个巧妙的拦截来用拨码开关进行输入，才有了现在的程序。

使用的汇编代码由网页的 Compiler Explorer 生成，然后翻译为机器码，这对于我来说比在本地配置 RISC-V 工具链更方便，也不需要使用虚拟机。但是 Compiler Explorer 生成的代码在使用前还需要进行专门的修改，有点麻烦。

最开始实现 CPU 的时候，我的时钟接反了好几次，折腾了半天才弄好。

综合后输出了一个文件 `tight_setup_hold_pins`，提醒我有很多地方的时序都很紧张，这可能就是单周期 CPU 的一个主要缺陷吧。